

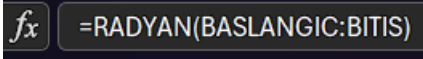
Veri Yapıları Lisans Proje Ödevi

Öncelikle bize verilen Excel tablosundaki 22 satır veriden özgün Hash değerleri elde edebilmek için bize verilen verileri bir dizi işlemten geçirdim. İzlediğim işlemler sırasıyla şu şekilde:

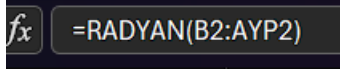
1. Adım

Veri satırlarında bulunan her bir -1,5 -0,5 0,5 ve 1,5 elemanlarını birer geometrik açı olarak varsaydım ve bu açıların Radyan cinsinden karşılığını hesapladım.

Her bir satır için ilgili formül şu şekildedir:



Örnek olarak A Etiketinde bulunan 1. Veri satırına ait formül şu şekildedir:



Burada B2 Hücresi verinin başlangıcını ve AYP2 Hücresi verinin sonunu göstermektedir.

Formülün çıktısı ise her bir veri satırında bulunan

-1,5 değerleri için -0,03,

-0,5 değerleri için -0,01,

0,5 değerleri için 0,01,

1,5 değerleri için 0,03

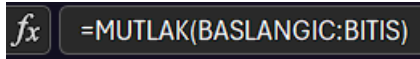
Değerlerini üretmektedir.

Bu işlemi Verilerimizi içeren Excel Dosyasında bulunan 22 Satırın her biri için ayrı ayrı uyguladım.

2. Adım

Radyan Cinsinden elde ettiğim değerlerin bazıları pozitif, bazıları ise negatif değere sahip. Hepsini tek bir çatı altında toplayabilmek için elde ettiğim tüm değerlerin mutlak değerini hesapladım.

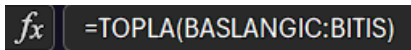
İlgili fonksiyon şu şekildedir:



Bu fonksiyon bize elde ettiğimiz radyan değerlerinin tümünün pozitif olarak elimizde bulunmasını sağlayacak.

3. Adım

Pozitif Radyan cinsinden elde ettiğim tüm verileri her bir satır kendi içerisinde ayrı olacak şekilde topladım. Bunun için ilgili fonksiyon şu şekildedir:



Böylece her bir veri satırı için toplam pozitif radyan değerini elde etmiş olacağız.

Bu üç adımın sonunda elde edeceğimiz Excel tablosu şu şekilde görünecek:

	A	B
1		
2	A grubu	22,73
3	A grubu	18,54
4	A grubu	19,26
5	A grubu	20,13
6	A grubu	22,13
7	A grubu	17,85
8	A grubu	14,15
9	A grubu	19,37
10	A grubu	20,8
11	A grubu	17,39
12	A grubu	10,76
13	A grubu	4,76
14	A grubu	14,98
15	A grubu	15,29
16	A grubu	3,48
17	A grubu	9,28
18		
19		
20		
21		
22	B grubu	8,09
23	B grubu	72,47
24	B grubu	61,48
25	B grubu	68,22
26	B grubu	71,14
27	B grubu	68,36

4. Adım: *Kod Yazma Aşaması.*

Yazılım aşamasının kolay olması açısından Excelde elde ettiğim son değerleri doğrudan hashValues adında bir dizide saklamaya karar verdim.

```
//Excel üzerinde gerçekleştirdiğim işlemler sonucu elde ettiğim değerleri hashValues adındaki diziye aktardım.  
float hashValues[22] = { 22.73 , 18.54 , 19.26 , 20.13 , 22.13 , 17.85 , 14.15 , 19.37 , 20.80 , 17.39 , 10.76, 4.76, 14.98 , 15.29 , 3.48 , 9.28,  
                        8.09 , 72.47 , 61.48 , 68.22 , 71.14 , 68.36 };
```

Elde edilen değerleri HashTablosuna yerleştirirken bağlı liste yapısından yararlanabilmek için bir node oluşturdum.

```
struct node {  
    float deger;  
    node* next;  
};
```

hashFonksiyonu

Şimdi ise sırada HashTablosunu oluştururken elemanları doğru yerlere yerleştirmemizi sağlayacak olan Hash Fonksiyonunu oluşturmakta.

HashFonksiyonu birinci parametre olarak Önceki adımlarda elde ettiğimiz değerleri saklayan hashValues dizisini alıyor.

Fonksiyon İkinci Parametre olarak ise verilerimizi yerleştirecek olduğumuz HashTablosunu alıyor.

Fonksiyonun her bir değer için birer kez tekrarlanabilmesi adına bir for döngüsünden oluşuyor.

index değişkeni hashValues dizisinde bulunan her bir değer 40'a bölünmesiyle 0 ya da 1 olacak şekilde değer alıyor ve verileri

HashTablosunda yerleştireceğimiz indisleri belirliyor.

Sağ tarafta görülen Hash Fonksiyonuna ait kod bloğu elde ettiğimiz Hash değerlerini Hash Tablosuna yerleştirirken bağlı liste yapısını kullanabilme kolaylığı sağlamak için root adında bir düğüm işaretçisi oluşturuyor ve elimizdeki hashValue değerlerini deger parametresine atıyor.

```
node* root = new node();  
root->deger = values[i];  
root->next = NULL;
```

```
if (dizi[index] == NULL) {  
    dizi[index] = root;  
}  
  
else if (dizi[index]->next == NULL) {  
    dizi[index]->next = root;  
}  
  
else {  
    node* temp = dizi[index];  
    while (temp->next != NULL)  
        temp = temp->next;  
    temp->next = root;  
}
```

Solda görülen kod bloğu ise verileri Hash Tablosuna yerleştirirken Hash Fonksiyonundan elde edilen indislerin uygunluk durumunu kontrol ediyor ve indislerde bulunan eleman sayısına göre bağlı listenin sonuna ekleme yapıyor.

Eğer 0 veya 1 indislerinde hiç eleman yoksa root işaretçisini doğrudan bu indislere atıyor.

Eğer 0 veya 1 indislerinde yalnızca bir eleman varsa ilk elemanın next'ine yani ikinci elemana root işaretçisini atıyor.

Bu iki özel durum dışında herhangi bir sayıda eleman var ise temp adında bir işaretçi oluşturarak dizinin ilgili indisindeki ilk düğümden başlayarak boş olmayan son düğüme kadar bağlı listeyi dolaşıyor ve son düğüme geldiğinde bir sonraki elemana root işaretçisini atıyor.

```
void yazdir(node* dizi[]) { //Sonuçları ekranda daha rahat görüntüleyebilmek için yazdir fonksiyonunu tanımladım.
//Parametre olarak HashTablosunu temsil eden bir dizi alıyor.
for (int i = 0; i <= 1; ++i) {
    node* root = dizi[i];
    int sayac = 1;
    //root işaretçisi 0 ve 1 indislerinin başından başlayıp NULL değere ulaşana kadar dizi indislerinde
    { //Bulunan bağlı listeleri dolaşarak her bir değeri ekrana yazdırıyor.
        while (root != NULL) {

            if (i == 0) {
                cout << "A[" << sayac++ << "] = " << root->deger << endl;
            }
            else { //İşaretçinin dolaştığı bağlı liste 0 indisinde ise ekrana A
                cout << "B[" << sayac++ << "] = " << root->deger << endl; //İşaretçinin dolaştığı bağlı liste 1 indisinde ise ekrana B
            } //Yazdırıyor.
            root = root->next;
        }
    }
}
```

yazdir Fonksiyonu

Hash tablosuna yerleştirilen değerleri kolay ve anlaşılır bir biçimde görüntüleyebilmek için yazdir fonksiyonunu geliştirdim. Tek bir parametre olarak Hash Tablosunu temsil eden elemanları node* tipinde bir dizi alıyor.

```
for (int i = 0; i <= 1; ++i) {
    node* root = dizi[i];
    int sayac = 1;
```

Hash Tablosu 0 ve 1 olmak üzere yalnızca iki indisten oluştuğu için fonksiyon içerisindeki for döngüsü yalnızca iki kez tekrar ediyor.

Her bir indis için içerisinde bulunan bağlı listelerin ilk düğümünü gösteren bir root işaretçisi oluşturuyor ve elemanları sayabilmek için 1'den başlayan bir sayaç değişkeni ataması yapıyor.

```
while (root != NULL) {

    if (i == 0) {
        cout << "A[" << sayac++ << "] = " << root->deger << endl;
    }
    else { //İşaretçinin dolaştığı bağlı liste 0 indisinde ise ekrana A
        cout << "B[" << sayac++ << "] = " << root->deger << endl; //İşaretçinin dolaştığı bağlı liste 1 indisinde ise ekrana B
    } //Yazdırıyor.
    root = root->next;
}
```

Fonksiyon içerisinde bulunan while döngüsü ise root işaretçisi NULL olana kadar yani bir diğer deyişle işaretçi bağlı listenin sonuna ulaşana kadar bağlı listeyi dolaşıyor. Bu dolaşma sırasında bağlı liste içerisinde bulunan her bir eleman için sayac değişkenini bir arttırıyor ve sayaç değeri ile birlikte bu değeri ekrana yazdırıyor. Değerleri yazdırma sırasında satırların başına döngünün içinde bulunduğu Hash Tablosuna ait indis 0 ise A, 1 ise B etiketine ait olduğunu belirten A[sayaç] ya da B[sayaç] ibaresini ekliyor.

Program çalıştırıldığında oluşacak Hash Tablosuna ait çıktı şu şekildedir:

<i>A Etiketi</i>	<i>B Etiketi</i>
22,73	72,47
18,54	61,48
19,26	68,22
20,13	71,14
22,13	68,36
17,85	
14,15	
19,37	
20,8	
17,39	
10,76	
4,76	
14,98	
15,29	
3,48	
9,28	

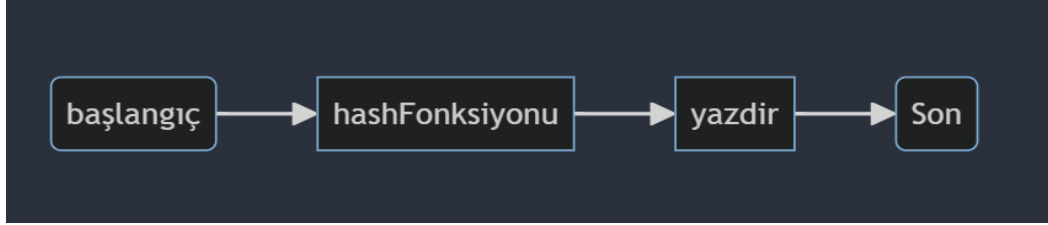
8,09

Tablodan görülebileceği üzere 16 adet A verisi ve 6 Adet B verisinden Tüm A verileri başarılı bir şekilde A etiketine, 6 adet B verisinden ise 5 Tanesi başarılı bir biçimde B etiketine yerleştirilmiştir. Programın başarısız şekilde yaptığı tek yerleştirme ise B grubuna ait verilerden elde edilen 8.09 değeridir.

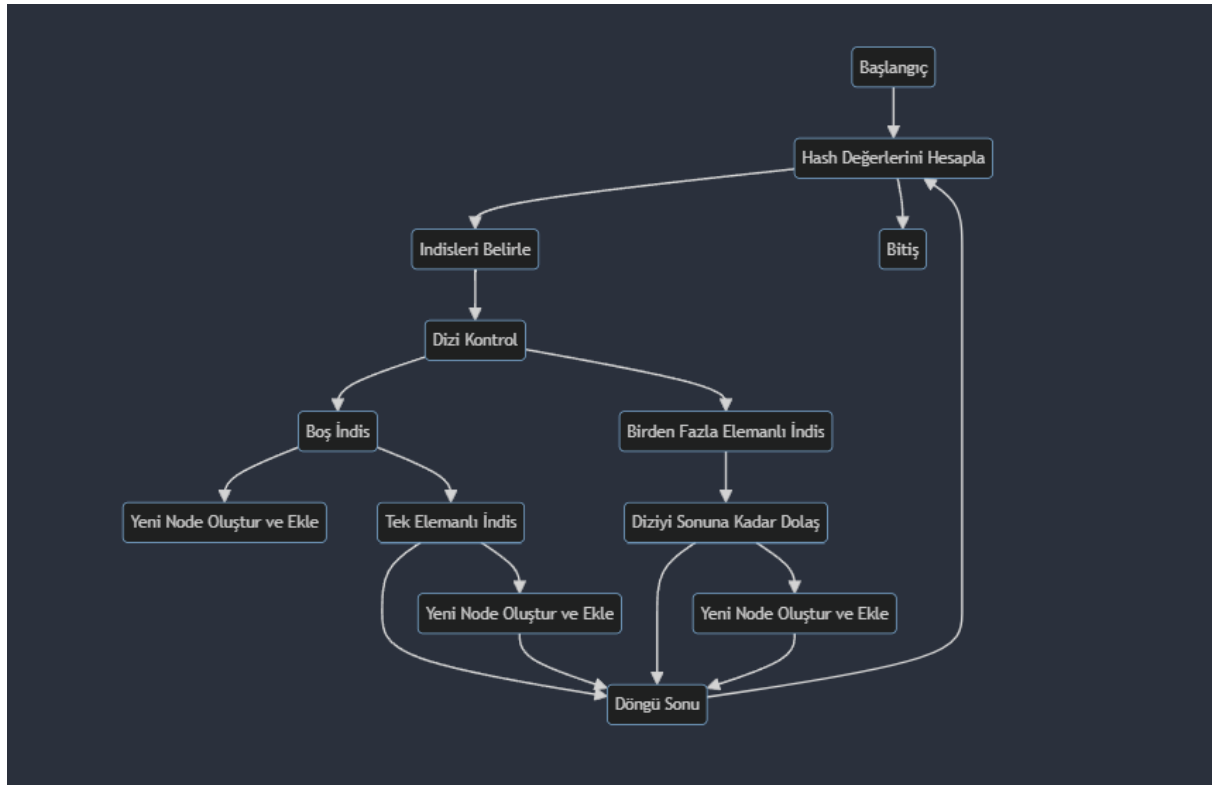
Programın yazılı akış şeması şu şekildedir:

1. **Başla:** İşlem başlar
2. **Ana Fonksiyon (main):** Ana fonksiyon başlar.
3. **Hash Tablosu Oluşturma:** Ana fonksiyon, **hashFonksiyonu** fonksiyonunu çağırarak hash tablosunu oluşturur.
4. **Hash Tablosunu Yazdırma:** Ana fonksiyon, **yazdir** fonksiyonunu çağırarak hash tablosunu ekrana yazdırır.
5. **Hash Fonksiyonu (hashFonksiyonu):** **hashFonksiyonu** fonksiyonu başlar.
6. **Hash Değerleri İşleme:** **hashFonksiyonu**, verilen hash değerlerini işler ve hash tablosuna ekler.
7. **Dizi Indis Hesaplama:** Her bir hash değerinin uygun dizi indisini hesaplar.
8. **Yeni Node Oluşturma:** Her bir hash değeri için yeni bir node oluşturur.
9. **Diziye Node Ekleme:** Oluşturulan node'ları uygun dizi indisine ekler.
10. **Bitir:** İşlem tamamlanır.

Programın Görsel akış şeması ise şu şekildedir:



hashFonksiyonu fonksiyonuna ait akış diyagramı:



yazdir Fonksiyonuna ait akış diyagramı:

